

MIAE 215: Computer Programming for Engineers

Master Step-by-Step Code Execution & Trace Manual

Concordia University · Department of Mechanical, Industrial & Aerospace Engineering (MIAE)

Standard: Universal Step-by-Step Expansion Method (exact_ode_step_by_step.pdf standard applied to C++)

The Step-by-Step Execution Methodology

Every programming and code trace problem follows the universal 4-pillar structure:

1. **Numbered Step Breakdown:** Every line of code and phase is tagged (Step 1 , Step 2 , Step 3 ...).
 2. **Theory Before Execution:** The C++ language standard, operator precedence hierarchy, data type rule, or IEEE 754 floating-point standard is stated before numbers are computed.
 3. **Zero Skipped Calculations:** All operator evaluations, memory writes, and type coercions are shown.
 4. **"Pattern to Remember" Box:** A concise algorithmic rule box at the end of each problem.
-

Table of Contents

- [Problem 1: Integer Truncation & Operator Precedence](#)
 - [Problem 2: Floating-Point Division by Zero & IEEE 754 Special Values](#)
 - [Problem 3: Machine Epsilon & Double Precision Limits](#)
 - [Problem 4: Trace Table for While-Loops & Compound Accumulators](#)
 - [Problem 5: Pass-by-Value vs. Pass-by-Reference Memory State Tracking](#)
-

Problem 1: Integer Truncation & Operator Precedence

Problem Statement

Determine the exact values of variables `q` and `x` and explain what prints to the screen:

```
int q;  
double x;  
q = 7 - (7 / 3) * 3;  
x = 1 / 3 * 10.0;  
cout << q << x;
```

Step 1: Analyze types and operator precedence for `q`

- `q` is of type `int`.

- Expression: `q = 7 - (7 / 3) * 3;`
- Parentheses `()` have the highest precedence, followed by multiplication/division `(*, /)`, followed by subtraction `(-)`.

Step 2: Evaluate the sub-expression `(7 / 3)`

- Both operands `7` and `3` are integer literals (`int`).
- In C++, integer division truncates towards zero (drops any fractional part):

$$7/3 = 2.3333\ldots \implies 2$$

Step 3: Perform multiplication and subtraction

- Multiply: $2 * 3 = 6$.
- Subtract: $7 - 6 = 1$.
- Store in `q`: `q = 1`.

Step 4: Analyze operator precedence and type coercion for `x`

- `x` is of type `double`.
- Expression: `x = 1 / 3 * 10.0;`
- Operators `/` and `*` share equal precedence and associate **left to right**.
- Therefore, `1 / 3` is evaluated first!

Step 5: Evaluate `1 / 3 * 10.0`

- `1 / 3` is integer division: $1/3 = 0.3333\ldots \implies 0$.
- Next, `0 * 10.0` converts `0` to `double` (`0.0 * 10.0 = 0.0`).
- Store in `x`: `x = 0.0`.

Step 6: Determine console output

- `cout << q << x;` outputs `1` immediately followed by `0` without any separator space.
- **Console Output:** `10`

★ C++ Precedence Pattern to Remember

1. In C++, integer divided by integer ALWAYS yields an integer (truncates toward zero).
 2. To get a floating-point result, at least one operand must be float/double (e.g. `1.0 / 3`).
 3. Left-to-right associativity evaluates `1 / 3` before multiplying by `10.0`.
 4. Continuous `cout << a << b` streams outputs without spaces unless explicit whitespace (`" "`) is provided.
-

Problem 2: Floating-Point Division by Zero & IEEE 754 Special Values

Problem Statement

Determine the output and machine state of:

```
double z = 0.0;
double y = -1.0 / z;
cout << y << endl;
```

Step 1: Identify data types and the operation

- Variable `z` is a 64-bit IEEE 754 floating-point variable (`double`) initialized to `0.0`.
- The operation is `-1.0 / z`, which evaluates floating-point division by zero with a negative numerator.

Step 2: State the IEEE 754 floating-point standard rule

- Unlike integer division by zero (which causes an immediate runtime crash / hardware exception), IEEE 754 floating-point standard defines three special values:
 - $+\infty$ (`inf`): positive number divided by `0.0`.
 - $-\infty$ (`-inf`): negative number divided by `0.0`.
 - NaN (Not-a-Number): `0.0/0.0` or $\infty - \infty$.

Step 3: Compute the result

$$-1.0/0.0 = -\infty \implies \text{-inf}$$

Step 4: Console display

The standard C++ streams library outputs:

```
-inf
```

Problem 3: Machine Epsilon & Double Precision Limits

Problem Statement

Determine the values of `r3` and `r4`:

```
double r3 = 1.0 - (1.0 - 1.0e-15);
double r4 = 1.0 - (1.0 - 1.0e-20);
```

Step 1: State the precision limits of IEEE 754 64-bit `double`

- A standard IEEE 754 `double` allocates:
 - 1 sign bit
 - 11 exponent bits
 - 52 fraction (mantissa) bits
- This provides 53 bits of precision, equivalent to approximately **15 to 17 significant decimal digits**.
- Machine epsilon: $\epsilon_{machine} = 2^{-52} \approx 2.22 \times 10^{-16}$.
- Any delta added to 1.0 smaller than $\epsilon_{machine}/2$ is lost due to roundoff.

Step 2: Evaluate `r3`

- Delta is $1.0 \times 10^{-15} > \epsilon_{machine}$.
- $1.0 - 1.0 \times 10^{-15}$ is representable in memory as 0.9999999999999999.
- Subtracting from 1.0: $1.0 - 0.9999999999999999 = 1.0 \times 10^{-15}$ (`1e-15`).

Step 3: Evaluate `r4`

- Delta is $1.0 \times 10^{-20} \ll \epsilon_{machine}$.
- The gap is beyond the 53-bit mantissa width. In memory, $(1.0 - 1.0 \times 10^{-20})$ rounds strictly to 1.0.
- Subtracting: $1.0 - 1.0 = 0.0$.

Problem 4: Trace Table for While-Loops & Compound Accumulators

Problem Statement

Construct a complete trace table showing all variable states for:

```
int a = 2, b = 10;
while (a < b) {
    b -= a;
    a *= 2;
}
cout << a << " " << b << endl;
```

Step 1: Variable Initialization

- Memory state: `a = 2`, `b = 10`.

Step 2: Construct the iteration trace table

Iteration	Condition (a < b)	b -= a (b = b - a)	a *= 2 (a = a * 2)	New (a, b) State
0 (Init)	—	—	—	a = 2, b = 10
1	2 < 10 (True)	b = 10 − 2 = 8	a = 2 × 2 = 4	a = 4, b = 8
2	4 < 8 (True)	b = 8 − 4 = 4	a = 4 × 2 = 8	a = 8, b = 4
3	8 < 4 (False!)	(Loop terminates)	(Loop terminates)	a = 8, b = 4

Step 3: Output Formulation

The loop executes exactly 2 times and terminates when $a = 8$ is no longer strictly less than $b = 4$.

- Console Output: 8 4

Problem 5: Pass-by-Value vs. Pass-by-Reference Memory State Tracking

Problem Statement

Determine what prints:

```
void modify(int x, int& y) {
    x += 10;
    y += 20;
}

int main() {
    int a = 5, b = 5;
    modify(a, b);
    cout << a << " " << b << endl;
    return 0;
}
```

Step 1: Trace memory allocation in main()

- main() allocates stack addresses:
 - Variable a : holds value 5 .
 - Variable b : holds value 5 .

Step 2: Analyze parameter passing in modify(a, b)

- First parameter x : **Pass-by-value** (int x). A local copy of a is created in modify() with value 5 .
- Second parameter y : **Pass-by-reference** (int& y). y is an alias (direct reference) pointing to the memory address of b in main() .

Step 3: Execute `modify()` body

- `x += 10;` modifies only the local copy: $x = 5 + 10 = 15$. Variable `a` in `main()` is **unchanged** (`a = 5`).
- `y += 20;` directly modifies the referenced variable `b` in `main()`: $b = 5 + 20 = 25$.

Step 4: Output Formulation

Upon return to `main()`, `a` retains its original value `5`, while `b` reflects the modification `25`.

- **Console Output:** `5 25`

✚ Parameter Passing Pattern to Remember

1. `type var` : Pass-by-value $\implies$ changes inside the function do NOT affect the caller.
2. `type& var` : Pass-by-reference $\implies$ changes inside the function directly alter the caller's variable.
3. Trace variables using stack memory boxes to avoid exam confusion.